模块 5：Feedback — 评估与修正的策略空间

模块概述

在 AI 编程代理的循环中，Feedback 回答两个最关键的问题：

1. **怎么知道对不对？**（评估）
2. **错了怎么办？**（修正）

没有 Feedback，Agent 就是”蒙眼射箭”——即使 Grounding 做得再精确、Planning 再周密、Action 再有力，如果不能判断结果的正确性并在出错时调整，整个系统就会”一条路走到黑”。

回顾我们课程的核心循环图：

```
Grounding → Planning → Action → Grounding（察果）→ Feedback →（下一）
```

Feedback 环节本质上是一个**信号生成器**——它消费 Action 的产出，生成一个信号，这个信号会影响：

- 是否接受当前结果（accept / reject）
- 是否需要重试、回退、或重新规划
- 哪些信息应该被”记住”以改善未来尝试

本模块将系统梳理四种 Feedback 策略，从最可靠的外部验证到最灵活的人在回路，再深入 Claude Code 的 Hooks 系统和失败恢复决策框架。

策略全景

策略	信号来源	可靠性	覆盖面	延迟
外部验证	测试/Lint/类型检查	★★★★★	有限（取决于测试覆盖率）	秒级

策略	信号来源	可靠性	覆盖面	延迟
模型自评	LLM 自身	★★★	广（任何输出都可评估）	秒级
Reflexion	失败经验的语言记忆	★★★★	跨 episode 累积	分钟级
人在回路	人类判断	★★★★★	取决于人的专业度	分钟~小时级

5.1 策略 1：外部验证

核心思想

外部验证是 Feedback 中最可靠的信号来源。它的哲学很简单：**不要问 LLM 代码对不对，让代码自己证明自己。**

外部验证工具的共同特点是：

- **确定性**：同样的输入，同样的结果
- **客观性**：不受 prompt 或上下文影响
- **可复现**：任何人在任何时间运行都得到相同结论

5.1.1 测试执行：最权威的 Ground Truth

测试套件是判断代码是否正确的”终极仲裁者”。对 AI 编程代理来说，运行测试是最强的 Feedback 信号。

为什么测试如此重要？

```
Agent 修改了 auth.py 中的 validate_token() 函数
     ↓
⍰行 pytest tests/test_auth.py
     ↓
⍰⍰⍰果：3 passed, 1 failed
     ↓
失⍰信息：test_expired_token — AssertionError: expected False, got True
     ↓
Agent ⍰得了精确的、可操作的 Feedback
```

测试提供的不仅是”对/错”的二元信号，更重要的是**错误定位信息**：

- 哪个测试失败了（定位到功能点）
- 失败的断言是什么（期望值 vs 实际值）
- 堆栈追踪（定位到代码行）

Claude Code 中的测试执行

Claude Code 通过 Bash tool 执行测试命令。一个典型的交互流程：

```
用⍰：修复 login 接口的 bug

Claude Code 的内部⍰程：
1. [Grounding] ⍰取相关代⍰和⍰⍰文件
2. [Planning] 确定修复方案
3. [Action]   修改代⍰
4. [Feedback] ⍰行 `pytest tests/test_login.py -v`
5. 根据⍰⍰⍰果决定下一步
```

在 Claude Code 的 system prompt 中，有明确的指引鼓励运行测试来验证修改。当用户的项目中有 `CLAUDE.md` 文件指定了测试命令时（例如 `npm test`、`pytest`、`cargo test`），Claude Code 会优先使用这些命令来验证自己的修改。

实践建议：在 CLAUDE.md 中指定测试命令

```
# CLAUDE.md
## ⍰⍰
- ⍰行全部⍰⍰: `pytest`
- ⍰行⍰个模⍰⍰⍰: `pytest tests/test_<module>.py -v`
- ⍰行⍰覆盖率: `pytest --cov=src`
```

这让 Agent 无需猜测如何运行测试——降低了 Grounding 的成本，提升了 Feedback 的效率。

5.1.2 Lint 工具：代码质量的自动化审查

Lint 工具检查代码风格、常见错误模式和潜在的 bug。它们比测试”轻量”得多，但能捕捉到一类测试不容易发现的问题。

语言	常用 Lint 工具	典型检查项
Python	Ruff, Pylint, flake8	未使用变量、导入顺序、复杂度

语言	常用 Lint 工具	典型检查项
JavaScript/TypeScript	ESLint, Biome	未使用变量、类型安全、最佳实践
Go	golangci-lint	错误处理、竞态条件、性能
Rust	clippy	惯用写法、性能、正确性

Lint 作为 Feedback 的价值

```
# Agent 生成的代⍰
def process_data(data):
    result = []
    temp = data.copy()  # Lint 警告: temp ⍰⍰后未使用
    for item in data:
        if item > 0:
            result.append(item * 2)
    return result
```

Lint 工具（如 Ruff）会立刻指出 `temp` 是”dead code”——这不会导致测试失败，但它暗示 Agent 的推理过程出了问题（可能原本打算使用 `temp` 但忘记了）。

5.1.3 类型检查：静态的正确性保证

类型检查器在不运行代码的情况下，验证类型的一致性。

```
// Agent 生成的 TypeScript 代⍰
function getUserAge(user: User): string {
    return user.age;  // 类型⍰⍰: number 不能⍰⍰⍰⍰string
}
```

TypeScript 编译器（`tsc --noEmit`）或 Python 的 `mypy` 能在编译/检查阶段就发现这类错误。

类型检查的独特价值：跨文件一致性

当 Agent 修改了一个接口的签名，类型检查器能自动发现所有调用方的不一致：

```
修改: interface User { age: number → age: string }
    ↓
tsc ⍰⍰: 15 个文件中引用了 user.age 作⍰⍰number
```

```
    ↓
Agent 得了完整的影响范信息
```

这是测试和 Lint 都很难做到的——类型检查器天然理解代码之间的依赖关系。

5.1.4 构建系统：最终的集成验证

构建系统（`npm run build`、`cargo build`、`go build`）验证的是整个项目能否成功编译/打包。它是一种”粗粒度但全面”的验证。

```
Agent 修改了多个文件
    ↓
行 `npm run build`
    ↓
Build 失: Cannot find module './utils/helper'
    ↓
Agent 自己忘建一个依文件
```

5.1.5 外部验证的局限性

尽管外部验证是最可靠的 Feedback，但它有明确的局限：

1. **测试可能不存在**：很多项目的测试覆盖率不高，Agent 修改的代码可能恰好没有测试覆盖
2. **测试可能不覆盖变更**：Agent 修改了一个边界条件，但现有测试只覆盖了正常路径
3. **通过测试 ≠ 正确**：代码可能通过了所有测试，但引入了性能问题、安全漏洞或设计缺陷
4. **环境依赖**：某些测试需要特定的外部服务（数据库、API），在 Agent 的环境中可能无法运行
5. **耗时**：大型项目的完整测试套件可能需要数十分钟

关键洞察：外部验证提供的是”必要非充分”的保证。测试全部通过 ≠ 代码正确，但测试失败 = 代码一定有问题。这种不对称性决定了我们需要其他 Feedback 策略作为补充。

5.2 策略 2：模型自评

核心思想

当外部验证工具不可用或不够用时，一个自然的想法是：**让 LLM 自己检查自己的输出**。这就是模型自评（Self-Evaluation）。

5.2.1 Self-Refine 机制

Self-Refine（Madaan et al., 2023）是一个重要的研究成果，它展示了 LLM 可以通过迭代的自我反馈来改进输出质量。

核心流程

```
┌──────────────┐
│  Generate    │ ← 初始生成
└──────────────┘
       │
       ▼
┌──────────────┐
│  Feedback    │ ← 自我□估："□个□出有什么□□？"
└──────────────┘
       │
       ▼
┌──────────────┐
│   Refine     │ ← 基于反□改□："根据上述□□，改□□出"
└──────────────┘
       │
       ▼
□□□到停止条件？──否──→ 回到 Feedback
       │
   是
       ▼
□□□□出最□□果
```

三个关键步骤

1. **Generate**：模型生成初始输出（代码、文本、方案等）
2. **Feedback**：模型被要求评估自己的输出，指出存在的问题
3. **Refine**：模型根据自己指出的问题，生成改进后的版本

实现示例：代码审查自评

```
[第一□ - Generate]
Prompt: "□□一个函数，□算列表的移□平均"
Output:
    def moving_average(data, window):
        result = []
```

```
        for i in range(len(data) - window + 1):
            avg = sum(data[i:i+window]) / window
            return avg  # Bug: return 在循□内部
        return result

[第二□ - Feedback]
Prompt: "□□上面的代□，指出所有□□"
Output:
    "□□1: return avg 在 for 循□内部，□致只□算第一个窗口就返回
□□□□□□□2: 没有□理 window > len(data) 的情况
□□□□□□□3: 没有□理 window <= 0 的情况"

[第三□ - Refine]
Prompt: "根据上述□□，修正代□"
Output:
    def moving_average(data, window):
        if window <= 0:
            raise ValueError("Window must be positive")
        if window > len(data):
            raise ValueError("Window larger than data")
        result = []
        for i in range(len(data) - window + 1):
            avg = sum(data[i:i+window]) / window
            result.append(avg)
        return result
```

5.2.2 Self-Refine 在实际 Agent 中的体现

在 Claude Code 中，Self-Refine 并不以显式的三步循环出现，而是融入了多轮对话的自然流程。当 Claude Code 生成代码后遇到错误，它会：

1. 观察错误信息（来自 Bash tool 的输出）
2. 分析错误原因（隐式的 Feedback 步骤）
3. 修改代码（Refine 步骤）

这本质上就是 Self-Refine 的实例化——只不过第 1 步的 Feedback 信号来自外部（测试/编译错误），而不是纯粹的自我评估。

5.2.3 模型自评的有效场景

模型自评在以下场景中表现较好：

表面层面的错误 - 语法错误、拼写错误 - 明显的逻辑错误（如上面的 return 位置错误） - 风格不一致 - 遗漏的边界条件（当被明确要求检查时）

结构层面的问题 - 代码组织是否合理 - API 设计是否符合约定 - 是否遵循了项目的架构模式

5.2.4 模型自评的局限

模型自评何时不可靠?

1. **自我一致性偏差**（Self-Consistency Bias） 模型倾向于认为自己的输出是正确的。当被要求”检查这段代码是否有 bug”时，如果代码是模型自己写的，它更倾向于说”没有问题”。

2. **复杂逻辑错误** 对于涉及多步推理的微妙 bug，模型往往无法通过”再看一遍”发现问题:

   ```python
   def binary_search(arr, target):
       left, right = 0, len(arr) - 1
       while left <= right:
           mid = (left + right) // 2  # 潜在的整数溢出
           if arr[mid] == target:
               return mid
           elif arr[mid] < target:
               left = mid + 1
           else:
               right = mid - 1
       return -1
   ```

   `mid = (left + right) // 2` 在 Python 中不会溢出（因为 Python 的整数是任意精度的），但如果这是 Java 或 C++ 代码，就有溢出风险。模型可能会”知道”这个经典 bug，但对于**不常见的、项目特定的逻辑错误**，自评的效果很差。

3. **知识盲区** 模型无法评估自己不知道的东西。如果代码涉及模型训练数据中罕见的库或框架，自评的质量会显著下降。

经验法则：模型自评适合做”第一道防线”——快速检查明显问题。但不应作为唯一的 Feedback 机制，必须与外部验证结合使用。

5.2.5 提升自评质量的技巧

1. **角色分离**：让模型扮演”资深代码审查者”而非原作者

   ```
   "你是一位?格的代???者。以下代?由你的同事提交，?仔???……"
   ```

2. **checklist 驱动**：给出具体的检查清单

```
"□□□以下方面：
 1. □界条件是否□理？
 2. □□情况是否□理？
 3. 是否有□源泄漏□□？
 4. 并□安全性如何？"
```

3. **对比审查**：让模型比较修改前后的差异

```
"以下是修改前和修改后的代□差异，□□□□个□更是否引入了新□□"
```

5.3 策略 3：Reflexion

核心思想

Self-Refine 关注的是”在当前尝试中改进”。但如果 Agent 需要**多次尝试**才能完成任务呢？每次失败的经验能否被保留下来，避免重蹈覆辙？

这就是 Reflexion（Shinn et al., 2023）要解决的问题：**将失败经验转化为自然语言记忆，在后续尝试中参考。**

5.3.1 Reflexion 的三个组件

```
┌──────────────────────────────────────────┐
│              Reflexion Agent             │
│                                          │
│  ┌──────────┐   ┌──────────┐   ┌────────┐│
│  │  Actor   │   │Evaluator │   │  Self- │
│  │          │→  │          │→  │Reflect │
│  │(□行□作)  │   │(□估□果)  │   │(□□教□)│
│  └──────────┘   └──────────┘   └────────┘│
│       ↑                          │       │
│       │    ┌──────────────┐      │       │
│       └────│   Memory     │←─────┘       │
│            │ (□□存□)      │              │
│            └──────────────┘              │
└──────────────────────────────────────────┘
```

1. Actor（执行者）

Actor 是执行具体任务的模块——它读取当前状态和历史经验，生成动作序列。

2. Evaluator（评估者）

Evaluator 判断 Actor 的执行结果是否成功。它可以是： - 外部信号（测试通过/失败） - 模型判断（LLM 评估输出质量） - 环境反馈（编译错误、运行时异常）

3. Self-Reflection（自我反思）

这是 Reflexion 的核心创新。当任务失败时，Self-Reflection 模块不只是说”失败了”，而是生成一段**自然语言的经验总结**：

```
失□□□ #1:
□□□作: 直接修改 config.json 中的 database URL
□□□果: □□失□ — 其他服□也依□□个配置
  反思: "修改共享配置文件会影响其他服□。
        下次□□使用□境□量覆盖，而不是直接修改配置文件。"

失□□□ #2:
□□□作: 使用□境□量 DB_URL 覆盖配置
□□□果: □□失□ — □境□量名拼写□□
  反思: "□境□量名□□是 DATABASE_URL 而不是 DB_URL。
        下次□□先□□代□中□□使用的□境□量名。"

□□ #3（利用前两次的反思）:
□□□□: [不要直接修改配置文件] [□□□□使用的□境□量名]
□□□作: 先 grep 代□找到 DATABASE_URL，然后□置□境□量
□□□果: 成功！
```

5.3.2 与 Self-Refine 的关键区别

维度	Self-Refine	Reflexion
记忆范围	当前对话上下文	跨 episode 持久化
反馈粒度	“这个输出有什么问题”	“这次失败教会了我什么”
适用场景	单次生成的改进	多次尝试的任务
信息类型	具体的修改建议	抽象的经验教训

最核心的区别在于**记忆的持久性**。Self-Refine 的反馈在当前上下文中生效；Reflexion 的反思被存储下来，可以在新的尝试中被检索和使用。

5.3.3 Reflexion 在编程任务中的效果

Reflexion 论文在 HumanEval 编程基准测试上取得了显著改进：

```
HumanEval (pass@1):
  GPT-4 基⍰:       67.0%
  GPT-4 + Reflexion: 91.0%  (+24%)
```

这个提升非常显著——仅通过”让模型记住之前的失败经验”，就能将通过率提高近 24 个百分点。

为什么 Reflexion 在编程任务上特别有效？

1. **编程有明确的成功/失败信号**：测试通过/失败，这使得 Evaluator 非常可靠
2. **编程错误通常是可描述的**：错误信息本身就是高质量的反馈
3. **经验具有迁移性**：“注意空输入”这样的经验对类似任务也有用

5.3.4 Reflexion 思想在 Claude Code 中的映射

Claude Code 没有显式实现 Reflexion 的三组件架构，但 Reflexion 的思想在实际使用中有自然的映射：

隐式 Reflexion：上下文中的经验积累

当 Claude Code 在一次会话中多次尝试修复同一个问题时，之前的失败尝试和错误信息自然地保留在对话上下文中。模型可以”看到”之前的失败，并据此调整策略。

```
[⍰⍰1] Agent 修改了 parser.py → ⍰行⍰⍰ → 失⍰（⍰界条件）
[⍰⍰2] Agent 看到⍰⍰，修复⍰界条件 → ⍰行⍰⍰ → 失⍰（另一个⍰界条件）
[⍰⍰3] Agent 看到两次失⍰的模式，一次性⍰理所有⍰界条件 → ⍰⍰通⍰
```

这里的上下文窗口扮演了 Reflexion 中 Memory 的角色。

显式 Reflexion：CLAUDE.md 作为持久化记忆

用户可以在 `CLAUDE.md` 中记录经验教训，让 Agent 在每次会话中都能获取：

```
# CLAUDE.md
## 已知⍰⍰
```

- 修改 auth 模□□，一定要同□更新 middleware 中的 token □□
- 数据□迁移前先□份，使用 `pg_dump` 而不是自定□脚本
- 前端□□需要先□□□mock server: `npm run mock`

这本质上就是”人工 Reflexion”——人类将失败经验编码为自然语言，持久化存储，供 Agent 在后续会话中参考。

5.3.5 实现一个简单的 Reflexion 循环

以下伪代码展示如何在自己的 Agent 中实现 Reflexion：

```
class ReflexionAgent:
    def __init__(self, llm, max_attempts=3):
        self.llm = llm
        self.max_attempts = max_attempts
        self.memory = []  # 存□反思□□

    def solve(self, task: str) -> str:
        for attempt in range(self.max_attempts):
            # Actor: 生成解决方案（□上□史□□）
            context = self._build_context(task)
            solution = self.llm.generate(context)

            # Evaluator: □行□□
            success, feedback = self._evaluate(solution)

            if success:
                return solution

            # Self-Reflection: 从失□中提取教□
            reflection = self.llm.generate(
                f"任□: {task}\n"
                f"□□的方案: {solution}\n"
                f"失□原因: {feedback}\n"
                f"□□□□次失□的教□，用一句□概括□□避免什么、"
                f"下次□□注意什么。"
            )
            self.memory.append(reflection)

        return "□到最大□□次数，任□未完成"

    def _build_context(self, task: str) -> str:
        ctx = f"任□: {task}\n"
        if self.memory:
            ctx += "之前的□□教□:\n"
            for i, m in enumerate(self.memory):
                ctx += f"  教□{i+1}: {m}\n"
        ctx += "□生成解决方案:"
        return ctx

    def _evaluate(self, solution: str) -> tuple[bool, str]:
        # □行□□或其他外部□□
```

```
    result = run_tests(solution)
    return result.passed, result.error_message
```

5.4 策略 4：人在回路

核心思想

前三种策略都是”自动化”的——无论是运行测试、模型自评还是 Reflexion，都不需要人类参与。但有些情况下，**只有人类才能做出正确判断**。

人在回路（Human-in-the-Loop, HITL）是 Feedback 的”最后一道防线”，也是最灵活、最昂贵的策略。

5.4.1 什么时候需要人类介入？

1. 需求模糊

```
用?: "?化一下?个接口的性能"
    → ?化到什么程度？延?降低 50%？吞吐量翻倍？
    → 可以改??API 接口?？?是只能改内部??？
    → 可以引入?存?？?数据一致性有什么要求？
```

当 Agent 面对模糊的需求时，最好的 Feedback 就是”问人”。试图猜测用户的意图，往往会导致做了大量工作后被全部推翻。

2. 破坏性操作

```
Agent: "我需要?除 users 表中的所有??数据"
Agent: "我需要 force push 到 main 分支"
Agent: "我需要修改生??境的配置"
```

这些操作的共同特点是**不可逆或影响范围大**。一旦执行错误，代价极高。

3. 低置信度决策

```
Agent 内部状?:
 方案A: 重?整个模?（工作量大，但更?雅）
```

```
方案B：打□丁（快速，但增加技□□）
置信度：50/50，无法判断用□偏好
```

当 Agent 对多个方案的置信度相当时，交给人类决策是最高效的。

4. 超出能力范围

- 涉及商业决策（“这个功能是否值得开发？”）
- 涉及团队协调（“这个改动需要其他团队的配合”）
- 涉及模型知识边界（使用了模型不了解的内部系统）

5.4.2 Claude Code 的权限确认设计

Claude Code 的权限系统是 Human-in-the-Loop 的一个精心设计的实现。

分层权限模型

```
安全□别     操作类型          是否需要确□
──────────────────────────────────────────────
低□□□□□□□□□□□取文件          自□允□
低□□      搜索代□（Grep/Glob） 自□允□
中□□□□□□□□□□□文件          取决于权限模式
中□□□□□□□□□□□行□□/lint          取决于权限模式
高□□□□□□□□□□□行任意 Bash 命令   默□需要确□
高□□      安装依□          默□需要确□
高□□□□□□□□□□Git push               需要确□
```

权限模式

Claude Code 提供了多个权限模式，让用户根据信任度选择：

- **默认模式**：大部分操作需要确认
- **接受编辑模式**：文件编辑自动允许，危险操作仍需确认
- **宽松模式**：更多操作自动允许

这种设计的智慧在于：它不是简单的”全部允许”或”全部拒绝”，而是根据操作的风险等级做分级处理。

5.4.3 打断的代价 vs 错误的代价

人在回路的核心权衡是：

```
打断人类的成本              vs          ⍰⍰行⍰的成本
────────────────                        ──────────────────────────
- 中断用⍰的工作流                       - ⍰除了重要文件
- 等待用⍰响⍰的延⍰                      - 推送了有 bug 的代⍰到主分支
- 降低 Agent 的自主性体⍰                - 安装了⍰意依⍰
                                          - 泄露了敏感信息
```

关键原则:

> 对于可逆操作，偏向自主执行（降低打断成本）。 对于不可逆操作，偏向询问确认（降低错误成本）。

5.4.4 设计有效的人类检查点

好的 Human-in-the-Loop 设计需要考虑:

1. 提供充分的上下文

```
❌ 不好: "是否⍰行此操作？[Y/N]"
✅ 好:   "我将⍰除 3 个⍰⍰的数据⍰迁移文件:
           - 001_init.sql (2022-01-01)
           - 002_add_users.sql (2022-03-15)
           - 003_add_roles.sql (2022-06-20)
⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰些迁移已⍰⍰用到生⍰⍰境。确⍰⍰除？[Y/N]"
```

2. 提供可选方案

```
"我可以用两种方式修复⍰个 bug:
 A) 在⍰用方添加空⍰⍰⍰（改⍰小，但不⍰底）
 B) 重⍰底⍰函数使其不返回空⍰（改⍰大，但更根本）
 推荐方案 A，因⍰当前是 hotfix ⍰景。⍰⍰哪个？"
```

3. 批量化确认

避免每一步都询问，而是在关键节点批量确认:

```
❌ 不好: 修改每个文件都⍰一次
✅ 好:   "我⍰划修改以下 5 个文件来完成重⍰:
           1. src/auth/token.py — 修改 validate() ⍰名
           2. src/auth/middleware.py — 适配新⍰名
           3. src/api/routes.py — 适配新⍰名
           4. tests/test_auth.py — 更新⍰⍰
           5. docs/api.md — 更新文档
       是否⍰⍰？"
```

5.5 Hooks 系统

核心概念

Claude Code 的 Hooks 系统允许用户在 tool 调用的**前后**插入自定义脚本，实现自动化的 Feedback 机制。

如果说前面的策略是 Agent “内部”的 Feedback 机制，那么 Hooks 就是**用户为 Agent 安装的”外部传感器”**。

5.5.1 Hooks 的工作原理

```
用⍰⍰求 → Agent 决定⍰用 Tool
                ↓
          ┌──────────────┐
          │  Pre-Hook    │ ← 在 Tool ⍰行前⍰行
          │ (可阻止⍰行)  │
          └──────────────┘
                ↓
          ┌──────────────┐
          │  Tool ⍰行    │
          └──────────────┘
                ↓
          ┌──────────────┐
          │  Post-Hook   │ ← 在 Tool ⍰行后⍰行
          │ (可⍰⍰⍰果)  │
          └──────────────┘
                ↓
          Agent ⍰⍰推理
```

Pre-Hooks：在 tool 执行前触发 - 可以阻止执行（返回非零退出码） - 可以记录/审计操作 - 可以执行前置检查

Post-Hooks：在 tool 执行后触发 - 可以验证执行结果 - 可以执行补充操作（如格式化、测试） - 可以发送通知

5.5.2 配置方式

Hooks 在 Claude Code 的 settings 文件中配置（`~/.claude/settings.json` 或项目级别的 `.claude/settings.json`）：

```
{
  "hooks": {
    "PreToolUse": [
```

```
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/pre-check.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/post-validate.sh"
          }
        ]
      }
    ]
  }
}
```

配置要素:

- `PreToolUse` / `PostToolUse`： hook 触发的时机
- `matcher`： 匹配 tool 名称的正则表达式（`Bash`、`Write`、`Edit`、`Read` 等）
- `command`： 要执行的命令

5.5.3 Hook 脚本接收的信息

Hook 脚本通过 **stdin** 接收 JSON 格式的上下文信息:

```
{
  "tool_name": "Write",
  "tool_input": {
    "file_path": "/project/src/utils.py",
    "content": "..."
  }
}
```

Hook 脚本可以:

- **读取 stdin** 获取 tool 调用的详细信息
- **输出到 stdout** 将信息反馈给 Agent（Agent 会在对话中看到这些输出）

- **返回非零退出码**（仅 Pre-Hook）阻止 tool 执行

5.5.4 实用 Hook 示例

示例 1：文件写入后自动格式化

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "python3 /scripts/auto-format.sh"
          }
        ]
      }
    ]
  }
}
```

`auto-format.sh`：

```
#!/bin/bash
# 从 stdin 取 tool 入，提取文件路径
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path')

# 根据文件展名格式化工具
case "$FILE_PATH" in
    *.py)
        ruff format "$FILE_PATH" 2>&1
        ;;
    *.ts|*.tsx|*.js|*.jsx)
        npx prettier --write "$FILE_PATH" 2>&1
        ;;
    *.go)
        gofmt -w "$FILE_PATH" 2>&1
        ;;
esac
```

示例 2：编辑后自动运行相关测试

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
```

```
          {
            "type": "command",
            "command": "python3 /scripts/auto-test.py"
          }
        ]
      }
    ]
  }
}
```

`auto-test.py`：

```python
#!/usr/bin/env python3
import json
import subprocess
import sys
import os

input_data = json.load(sys.stdin)
file_path = input_data["tool_input"]["file_path"]

# 只?源代?文件?行??
if not file_path.endswith((".py", ".ts", ".js")):
    sys.exit(0)

# Python: ?找??的??文件
if file_path.endswith(".py"):
    base = os.path.basename(file_path)
    test_file = f"tests/test_{base}"
    if os.path.exists(test_file):
        result = subprocess.run(
            ["pytest", test_file, "-v", "--tb=short"],
            capture_output=True, text=True
        )
        # ?出到 stdout，Agent 会看到?些信息
        print(result.stdout)
        if result.returncode != 0:
            print(result.stderr)
```

示例 3：阻止危险的 Bash 命令（Pre-Hook）

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python3 /scripts/bash-guard.py"
          }
        ]
      }
    ]
```

```
  }
}
```

`bash-guard.py`：

```python
#!/usr/bin/env python3
import json
import sys

input_data = json.load(sys.stdin)
command = input_data["tool_input"].get("command", "")

# ⍰⍰危⍰命令模式
dangerous_patterns = [
    "rm -rf /",
    "DROP TABLE",
    "DROP DATABASE",
    "> /dev/sda",
    "mkfs",
    ":(){:|:&};:",  # fork bomb
]

for pattern in dangerous_patterns:
    if pattern.lower() in command.lower():
        # ⍰出警告信息⍰⍰Agent
        print(f"⚠️ 危⍰命令被阻止: ⍰⍰到 '{pattern}'")
        # 非零退出⍰阻止⍰行
        sys.exit(1)

# 正常退出，允⍰⍰行
sys.exit(0)
```

示例 4：通知 Hook — 关键操作完成时发送通知

```json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash /scripts/notify-on-deploy.sh"
          }
        ]
      }
    ]
  }
}
```

`notify-on-deploy.sh`：

```bash
#!/bin/bash
INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command')

# 如果命令包含部署相关操作，⍰送通知
if echo "$COMMAND" | grep -qE "(git push|deploy|kubectl apply)"; then
    echo "🚀 部署操作已⍰行: $COMMAND"
    # 可以集成 Slack/Discord webhook
    # curl -X POST "$SLACK_WEBHOOK" -d "{\"text\": \"Deploy: $COMMAND\"}"
fi
```

5.5.5 Hooks 的设计理念

Hooks 系统体现了一个重要的设计哲学：**将 Feedback 的控制权交给用户**。

与其让 Agent 自己决定何时运行测试、何时格式化代码，不如让用户通过声明式的配置来定义这些行为。这有几个优势：

1. **可预测性**：用户明确知道哪些操作会触发什么检查
2. **项目适配**：不同项目可以有不同的 Hook 配置
3. **低侵入性**：Hooks 不改变 Agent 的核心逻辑
4. **可组合性**：多个 Hooks 可以叠加使用

Hooks 本质上是把”被动的 Feedback”变成了”主动的、系统化的 Feedback”。

5.6 失败恢复策略

核心问题

当 Feedback 告诉 Agent “你错了”之后，下一步应该怎么做？

这不是一个简单的问题——不同的错误需要不同的恢复策略。盲目重试可能浪费时间，过早放弃可能错过正确答案。

5.6.1 四种恢复策略

```
⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰生
                |
```

```
                ▼
        ┌──────────────┐
        │ 是偶□性□□？  │──是──→ 重□（Retry）
        └──────────────┘
                │否
                ▼
        ┌──────────────┐
        │ 是局部□□？   │──是──→ 回退（Backtrack）
        └──────────────┘
                │否
                ▼
        ┌──────────────┐
        │ 是方案□□□？  │──是──→ 重新□划（Replan）
        └──────────────┘
                │否
                ▼
      上□人类（Escalate）
```

策略 1：重试（Retry）

适用场景：偶发性、非确定性的错误

```
例1: 网□超□
  Agent: curl https://api.example.com/data → timeout
 策略: 等待后重□，通常能成功

例2: 并□冲突
  Agent: git push → rejected (remote has newer commits)
 策略: git pull --rebase 后重□
```

注意：纯粹的”相同操作重试”在编程任务中很少有效。如果是代码逻辑错误，重试一百次结果也一样。重试主要适用于**环境层面**的偶发问题。

策略 2：回退（Backtrack）

适用场景：当前方向的局部实现有误，但整体方案正确

```
Agent 的修复□程:
 步□1: 修改 auth.py 中的 token □□□□  ✓
 步□2: 修改 middleware.py 适配新□□     ✗ (引入了新 bug)
 步□3: ???

回退策略:
  → 撤□步□2 的修改
  → 重新分析 middleware.py 的适配方式
  → 用不同的方法完成步□2
```

在 Claude Code 中，回退通常表现为： - 使用 Edit tool 撤销之前的修改 - 重新阅读相关代码，理解上下文 - 尝试不同的实现方式

策略 3：重新规划（Replan）

适用场景：当前方案从根本上行不通，需要换一个方向

```
原始□划：通□修改数据□□schema 来支持多租□
  → □□□行 → □□涉及太多表的迁移，□□太高

重新□划：
  → 放弃 schema 修改方案
  → 重新分析需求
  → 新方案：使用行□安全策略（Row-Level Security）
  → 改□范□大幅□小
```

Replan 的代价最高（放弃已有进度），但有时是唯一正确的选择。

何时触发 Replan?

- 连续多次 Backtrack 仍然失败
- 发现了初始 Planning 时未考虑到的约束
- 错误信息暗示根本方向有误

策略 4：上报人类（Escalate）

适用场景：Agent 已经穷尽了自动恢复能力

```
Agent: "我□□了 3 种方式修复□个 bug，但都失□了。
     根据□□信息，□□可能出在第三方□□xyz 的版本兼容性上。
     我不确定□□：
         A）降□□xyz 到 v2.x
         B）升□整个□目到 xyz v4.x 的 API
         C）□找 xyz 的替代□
□□□□□□□□□指□。"
```

上报时的质量决定了人类能否快速做出决策。好的上报应包含： - **已尝试的方案**和失败原因 - **当前的状态**（代码在什么状态，是否有未提交的修改） - **可选的下一步**和各自的利弊

5.6.2 失败恢复决策框架

将上述策略组合成一个实用的决策框架：

```
def decide_recovery(error, context):
    """
    决策失⍰恢复策略
    """
    # ⍰度1: ⍰⍰类型
    if error.is_transient():  # 网⍰超⍰、⍰冲突等
        return "retry", {"max_retries": 3, "backoff": "exponential"}

    # ⍰度2: ⍰⍰次数
    if context.attempt_count >= 3:
        if context.tried_different_approaches:
            return "escalate", {"reason": "多种方案均已失⍰"}
        else:
            return "replan", {"reason": "当前方案似乎不可行"}

    # ⍰度3: ⍰⍰范⍰
    if error.affects_only_latest_change():
        return "backtrack", {"undo": "latest_change"}

    if error.suggests_fundamental_issue():
        return "replan", {"reason": error.root_cause}

    # 默⍰: ⍰⍰回退最近的修改
    return "backtrack", {"undo": "latest_change"}
```

5.6.3 Claude Code 中的隐式恢复

Claude Code 没有显式的”恢复策略选择器”，但它通过模型的推理能力自然地实现了这些策略。

模型如何”自然恢复”？

当 Claude Code 执行一个操作并看到错误输出时，错误信息会成为下一轮推理的输入。模型基于错误信息的内容，自然地选择恢复策略：

```
⍰景: pytest ⍰出 "ModuleNotFoundError: No module named 'redis'"

模型的推理:
  "⍰⍰是缺少 redis 模⍰，⍰不是代⍰⍰⍰⍰⍰，
  而是依⍰缺失。我⍰⍰先安装 redis 包。"
  → ⍰式⍰⍰了 "修复⍰境⍰⍰" 策略

⍰景: pytest ⍰出 "AssertionError: expected [1,2,3], got [1,3,2]"

模型的推理:
```

```
  "□出□序不□，□明我的排序□□有□□。
□□□□我重新□□□sort 函数的□□。"
  → □式□□了 "Backtrack" 策略

□景：第三次□□不同的正□表□式仍然失□

模型的推理：
  "正□表□式方案似乎不适合□个解析需求。
  我□一个方案，使用 AST 解析□来□理。"
  → □式□□了 "Replan" 策略
```

这种”隐式恢复”的优势是灵活——模型可以根据具体情况做出细粒度的判断。劣势是不可控——用户无法确保模型一定会选择正确的恢复策略。

5.6.4 构建鲁棒的 Feedback 管道

综合本模块的所有策略，一个成熟的 AI 编程代理应该有这样的 Feedback 管道：

```
Agent □行 Action
       │
       ▼
┌──────────────────┐
│ 1. 外部□□        │  ← □行□□/lint/类型□□
│    (最高□先□)    │
└──────────────────┘
        │
        ▼  如果外部□□不可用或通□
┌──────────────────┐
│ 2. 模型自□       │  ← 模型□□自己的□出
│    (□充□□)       │
└──────────────────┘
        │
        ▼  如果□□□□
┌──────────────────┐
│ 3. 恢复决策      │  ← Retry / Backtrack / Replan
└──────────────────┘
        │
        ▼  如果多次恢复失□
┌──────────────────┐
│ 4. Reflexion     │  ← 存□教□到□□
└──────────────────┘
        │
        ▼  如果仍然无法解决
┌──────────────────┐
│ 5. 上□人类       │  ← 提供□□上下文
└──────────────────┘
```

Hooks 横跨整个管道，在任何环节都可以插入用户自定义的检查。

关键论文导读

Self-Refine (Madaan et al., 2023)

论文标题: Self-Refine: Iterative Refinement with Self-Feedback

核心贡献:

Self-Refine 证明了一个令人意外的结果：**同一个 LLM 可以通过迭代的自我反馈来显著改善自己的输出质量**，不需要额外的训练数据、额外的模型或强化学习。

方法论:

```
⍰入 x → LLM 生成初始⍰出 y₀
循⍰:
  f = LLM(y_t, "⍰个⍰出有什么⍰⍰？")      // Feedback
  y_{t+1} = LLM(y_t, f, "根据反⍰改⍰")     // Refine
 如果 f ⍰⍰足够好，或⍰到最大迭代次数，停止
⍰出 y_final
```

关键发现:

1. **跨任务通用**：Self-Refine 在代码生成、数学推理、对话响应等 7 个任务上都有效
2. **无需额外数据**：纯粹依赖模型自身的能力
3. **迭代收益递减**：通常 2-3 轮迭代后改进就会收敛
4. **模型越强效果越好**：强模型（如 GPT-4）的自评能力更准确，因此 Self-Refine 的提升更显著

局限性:

- 模型无法发现自己知识盲区内的错误
- 对于需要外部信息验证的任务（如事实核查），Self-Refine 可能会”自信地输出错误答案”
- 计算成本随迭代次数线性增长

对 Agent 设计的启示:

Self-Refine 告诉我们，即使没有外部验证信号，模型自评也是值得做的——但它的价值上限由模型的能力决定。在 Agent 设计中，Self-Refine 应该作为外部验证的**补充**，而不是替代。

Reflexion (Shinn et al., 2023)

论文标题: Reflexion: Language Agents with Verbal Reinforcement Learning

核心贡献:

Reflexion 提出了一种”语言强化学习”方法——不是通过梯度更新来学习，而是通过**自然语言的经验总结**来在多次尝试中持续改进。

核心创新：用语言替代梯度

传统的强化学习:

```
□作 → □励信号 → 梯度更新 → 模型参数改□
```

Reflexion:

```
□作 → □估□果 → 自然□言反思 → 存入□□ → 下次□□□参考
```

这个替代极其聪明:

- 不需要修改模型权重（不需要训练）
- 反思是”可解释的”（人类可以阅读和理解 Agent 学到了什么）
- 实现简单（只需要 prompt 和一个字符串列表作为记忆）

实验结果:

基准测试	基线	+ Reflexion	提升
HumanEval (pass@1)	80.1%	91.0%	+10.9%
MBPP	—	77.1%	—
AlfWorld	75%	97%	+22%

编程任务上的提升尤为显著——从 80.1% 到 91.0%。

为什么在编程上特别有效?

1. 编程有**明确的评估信号**（测试通过/失败），使得 Evaluator 非常可靠
2. 编程错误通常可以用**自然语言精确描述**（“忘了处理空列表”、“off-by-one 错误”）
3. 经验具有**迁移性**——一个任务中学到的教训往往对类似任务有帮助

Reflexion vs Fine-tuning

维度	Reflexion	Fine-tuning
是否修改模型	否	是
需要数据量	无（在线生成）	大量标注数据
可解释性	高（自然语言记忆）	低（模型黑盒）
泛化能力	中（依赖记忆检索）	高（内化到参数中）
实现复杂度	低	高

对 Agent 设计的启示:

Reflexion 启示我们：Agent 不需要”重新训练”就能”学习”——通过积累自然语言形式的经验，Agent 可以在多次尝试中持续改进。在实际产品中，这可以映射为：

- 用户级别的 `CLAUDE.md`：用户积累的项目经验
- 会话级别的上下文：当前会话中的失败经验
- 系统级别的 prompt：从用户群体中提炼的通用经验

实操环节

配置 Claude Code Hooks：自动运行测试

本实操的目标是配置 Claude Code 的 Hooks 系统，实现 **“每次编辑文件后，自动运行相关测试”**。

步骤 1：创建测试项目

首先准备一个带测试的示例项目：

```bash
mkdir -p ~/hook-demo/src ~/hook-demo/tests
```

创建 `~/hook-demo/src/calculator.py`：

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

创建 `~/hook-demo/tests/test_calculator.py`：

```python
import pytest
from src.calculator import add, subtract, multiply, divide

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_subtract():
    assert subtract(5, 3) == 2

def test_multiply():
    assert multiply(3, 4) == 12

def test_divide():
    assert divide(10, 2) == 5.0

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(1, 0)
```

步骤 2：创建 Hook 脚本

创建 `~/hook-demo/.claude/hooks/auto-test.sh`：

```bash
#!/bin/bash
# auto-test.sh — 在文件□□后自□□行相关□□
# 从 stdin □取 tool □用信息
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // .tool_input.file_path')
```

```
# 只⍰理 Python 源文件
if [[ "$FILE_PATH" != *.py ]]; then
    exit 0
fi

# 确定⍰目根目⍰
PROJECT_DIR=$(dirname "$FILE_PATH")
while [[ "$PROJECT_DIR" != "/" ]]; do
    if [[ -d "$PROJECT_DIR/tests" ]]; then
        break
    fi
    PROJECT_DIR=$(dirname "$PROJECT_DIR")
done

# 如果修改的是源代⍰文件，⍰找⍰⍰的⍰⍰文件
BASENAME=$(basename "$FILE_PATH" .py)
TEST_FILE="$PROJECT_DIR/tests/test_${BASENAME}.py"

if [[ -f "$TEST_FILE" ]]; then
    echo "🧪 自⍰⍰行⍰⍰: $TEST_FILE"
    cd "$PROJECT_DIR" && python -m pytest "$TEST_FILE" -v --tb=short 2>&1
else
    echo "ℹ️ 未找到⍰⍰的⍰⍰文件: $TEST_FILE"
fi


chmod +x ~/hook-demo/.claude/hooks/auto-test.sh
```

步骤 3：配置 Hooks

在项目目录下创建 `.claude/settings.json`：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/auto-test.sh"
          }
        ]
      }
    ]
  }
}
```

步骤 4：验证效果

在项目目录中启动 Claude Code，然后要求它修改 `calculator.py`：

```
你: "⍰⍰calculator.py 添加一个 power(base, exp) 函数"
```

观察 Claude Code 的行为：

1. Agent 使用 Edit/Write tool 修改 `calculator.py`
2. Post-Hook 自动触发，运行 `test_calculator.py`
3. 测试结果自动反馈给 Agent
4. 如果测试失败，Agent 可以立刻看到并修复

步骤 5：增强 — 添加格式检查 Hook

在 `.claude/settings.json` 中添加格式检查：

```json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/auto-test.sh"
          },
          {
            "type": "command",
            "command": "bash .claude/hooks/lint-check.sh"
          }
        ]
      }
    ]
  }
}
```

`lint-check.sh`：

```bash
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path')

if [[ "$FILE_PATH" == *.py ]]; then
    echo "🔍⍰⍰行 Ruff ⍰⍰..."
```

```
    ruff check "$FILE_PATH" 2>&1
fi
```

思考问题

完成实操后，思考：

1. 如果测试运行时间很长（>30 秒），Hook 会阻塞 Agent 多久？如何优化？
2. 如何设计 Hook 只运行”受影响的”测试，而不是所有测试？
3. Pre-Hook 和 Post-Hook 分别适合做哪些检查？

本模块小结

核心要点回顾

1. **Feedback 是 Agent 循环中不可或缺的一环**——没有评估和修正，Agent 就是”蒙眼执行”

2. **四种策略各有所长**：

   - **外部验证**最可靠，但覆盖有限
   - **模型自评**最灵活，但可靠性有限
   - **Reflexion**实现跨尝试的学习，但需要额外的记忆管理
   - **人在回路**最准确，但成本最高

3. **实际系统需要组合使用多种策略**——外部验证为主，模型自评为辅，Reflexion 提供记忆，人在回路兜底

4. **Hooks 系统**将 Feedback 的控制权交给用户，实现项目级别的自定义验证流程

5. **失败恢复不是简单的重试**——需要根据错误类型、尝试次数、错误范围来选择 Retry / Backtrack / Replan / Escalate

与前后模块的衔接

```
模□ 4（Action）                  模□ 5（Feedback）                 模□ 6（搜索策略）
────────────                    ──────────────                   ──────────────
Action □出□果          →        Feedback □估□果          →       搜索策略在更大的
Tool Use / CodeAct               外部□□ / 自□ / 人类        空□中□合
                                                                     Planning-Action-
Feedback
```

Feedback 是连接 Action 和下一轮 Planning 的桥梁。而在下一模块中，我们将看到如何将 Planning → Action → Feedback 这个循环本身作为”搜索过程”的一步，用搜索策略（如 MCTS）来系统化地探索解决方案空间。

思考题

1. **Self-Refine 的收敛性**：Self-Refine 通常在 2-3 轮后收敛。请思考：为什么更多的迭代不会带来更多改进？这与 LLM 的什么特性有关？

2. **Reflexion 的记忆管理**：如果 Reflexion Agent 运行了 100 次，积累了 100 条经验教训。请思考：如何避免记忆”膨胀”导致的 prompt 过长问题？哪些经验应该被”遗忘”？

3. **人在回路的粒度**：极端情况下——“每步都问人”和”从不问人”——哪个更差？为什么？如何找到最优的 Human-in-the-Loop 频率？

4. **Hooks 的性能影响**：假设你配置了 5 个 Post-Hooks（格式化 + Lint + 测试 + 安全扫描 + 文档生成），每次 Agent 编辑文件都会触发所有 Hooks。请分析：

   - 这对 Agent 的效率有什么影响？
   - 如何设计一个”智能调度器”来决定哪些 Hooks 需要运行？

5. **综合设计题**：你正在设计一个自动修复 GitHub Issue 的 Agent。请设计它的完整 Feedback 管道：

   - 使用哪些外部验证工具？
   - 何时触发模型自评？
   - 如何实现 Reflexion（经验存储在哪里？格式是什么？）
   - 什么条件下上报给人类？